

An application of XGBoost algorithm for online transaction fraud detection based on improved sailfish optimizer

Hongwei Chen^{1,a}¹School of Computer Science, Hubei University of Technology
Wuhan, ChinaLun Chen^{2,b*}²School of Computer Science, Hubei University of Technology
Wuhan, China* Corresponding author: C227919@126.com

Abstract—Fraud detection is relevant in this context because of the large scale of online transactions, which conceal a small number of fraudulent transactions. Machine learning (ML) fraud detection models have numerous hyperparameters of the eXtreme Gradient Boosting (XGBoost) algorithm, which are important for the performance impact of fraud detection models. In this paper, we propose an improved version based on Sailfish Optimizer (SFO) applied to online transaction fraud detection to optimize the hyperparameters of the XGBoost algorithm. Firstly, the optimization-seeking and local-optimal-skipping capabilities of Sailfish Optimizer are improved, and Improved Sailfish Optimizer (ISFO) is compared with other classical optimization algorithms, and the results show that the algorithm has outstanding. The Improved Sailfish Optimizer is then used to optimize the parameters of XGBoost (ISFO-XGBoost) and compared with several other machine learning classifiers such as: Decision Tree (DT), Random Forest (RF), XGBoost, and several optimization algorithms optimized for the XGBoost model. To validate the performance, a fraud dataset of online transactions is used for evaluation. The experimental results demonstrate that the proposed improved algorithm is highly promising in solving machine learning hyperparameter optimization challenges, and the ISFO-XGBoost algorithm achieves the best results.

Keywords: fraud detection, machine learning, hyperparameter, XGBoost, sailfish optimizer

I. INTRODUCTION

Online transactions have gained global popularity for their swiftness and convenience, and are gradually becoming the method of transaction for most people. According to the statistics of global credit card fraud transactions, the loss of credit card fraud is up to about \$28.65 billion as of 2019, and the loss is expected to reach about \$32.96 billion by 2023 [1]. Fraudulent transactions account for only a small fraction of the total transactions, and when faced with very few fraudulent transactions among tens of thousands of real transactions, manual screening becomes unattainable and detection of online fraudulent transactions becomes a very challenging problem.

Many methods have been researched in the field of fraud detection at this stage and the existing models use different techniques such as: machine learning, data mining and deep learning to detect fraudulent transactions [2]. Khatri et al. [3] used several machine learning algorithms for credit card fraud detection for comparison, KNN, LR, DT, RF and NB. Using a dataset generated from European cardholders in 2013, these models among which KNN has the best Sensitivity and

Precision in fraud detection. Trivedi et al. [4] try many supervised algorithms including RF and GB, the experimental results show that RF has better Accuracy and Precision.

Rajora et al. [5] conducted a comparative study of many machine learning algorithms for fraudulent transaction detection comparing RF and KNN and showed that RF has better Precision and AUC, but these studies did not address the problem of class imbalance that exists in the dataset used. The challenge of classifying datasets when class imbalance has been the subject of much research, and the solution to the problem is usually to use undersampling or oversampling techniques. Dornadula et al. [6] in order to deal with the high class imbalance in fraudulent transaction datasets problem, SMOTE sampling technique was used. Using DT, LR and IF as algorithms for fraud detection models, the results showed that IF is better than DT and LR for Precision.

The common methods to adjust algorithm hyperparameters include grid search [7] (GS) and random search [8] (RS). Putatunda et al. [9] used Hyperopt, GS, RS to adjust the hyperparameters of the XGBoost algorithm. Validated on six datasets, Hyperopt outperforms Accuracy and time performance compared to GS and RS. Qin et al. [10] analyzed the characteristics of Particle Swarm Optimization (PSO) to construct XGBoost based classification models. The hyperparameters of XGBoost were optimized using an adaptive PSO, which was higher than other base models in terms of evaluation metrics such as Precision and Recall.

Chen and Guestrin [11] proposed an advanced gradient augmentation algorithm, the extreme gradient augmentation (XGBoost) tree, which achieved good results in Kaggle competition data [12]. XGBoost in machine learning is used to implement fraud detection for online transactions, and the parameters of the XGBoost algorithm have an important effect on its detection performance. To deal with the numerous parameters of XGBoost, SFO [13] is improved to improve its parameter search process and enhance the handling of complex global searches. The fitness formula of the optimization function is defined so that the model finds the optimal hyperparameters and constructs the ISFO-XGBoost algorithm model for online transaction fraud detection.

In this paper, we compare the performance of ISFO-XGBoost with that of numerous classifiers widely used for fraud detection after processing imbalanced data using SMOTE sampling technique based on creditcard dataset. The experimental results show that the improved sailfish optimizer

has a strong optimization effect and the algorithm has a high classification accuracy.

In addition, this paper is divided into three parts: the second part improves the sailfish optimization algorithm and combines it with the XGBoost algorithm to construct an online fraud detection model. In the third part, the performance of the proposed model is evaluated with an online transaction dataset, and finally the fourth part summarizes the conclusions and future work.

II. MATERIALS AND METHODS

A. Sailfish Optimizer

Sailfish Optimizer(SFO) is a new nature-inspired metaheuristic optimisation algorithm inspired by a population of sardine hunting sailfish. SFO is excellent at improving the exploration and development phase, avoiding local optimisation, and converging at high speed especially for large scale global optimisation.

Sardines are similar to sailfish. Sailfish and sardines work together in concert to find the best adapted position in the solution space. The best position of the sailfish in the iteration is saved as an elite. The elite sailfish is the best positioned sailfish obtained so far and it can influence the sardine's manoeuvrability and acceleration during an attack. By saving the best position of the sailfish and saving the position of the wounded sardine in each iteration and using both as the best targets for the sailfish to hunt in concert.

Sailfish can increase their hunting success through coordinated attacks, with sailfish chasing and grazing their prey. Sailfish grazing behaviour is based on adjusting their position to the position of other sailfish around the sardine school. The SFO algorithm exhibits an alternating strategy of attack by sailfish during group hunting, in which the new position of the sailfish at the i iteration of $X_{new_SF}^i$.

$$X_{new_SF}^i = X_{elite_SF}^i - \lambda_i \times \left(rand(0,1) \times \left(\frac{X_{new_SF}^i + X_{injured_S}^i}{2} \right) - X_{old_SF}^i \right) \quad (1)$$

The sardine updates its position at each iteration based on the best position of the sailfish and its ability to attack. In the SFO algorithm, at the i th iteration, the sardine's new position $X_{new_S}^i$.

$$X_{new_S}^i = r \times (X_{elite_SF}^i - X_{old_S}^i + AP) \quad (2)$$

$X_{new_SF}^i$ is the position of the elite sailfish, $X_{old_SF}^i$ is the current position of the sardine. r is a random number of (0,1), is the attack strength of the sailfish at each iteration.

$$AP = A \times (1 - (2 \times Itr \times \varepsilon)) \quad (3)$$

A and ε are linear factors that reduce the value of energy attacks. AP is important, if the sailfish beat intensity is very high, all sardine positions will be updated.

In the final stages of the hunt, injured sardines that break free from the school are quickly captured. In the proposed algorithm, it is assumed that this occurs when the sardine is more suitable for predation than the sailfish. In this case, the

position of the sailfish replaces the position of the latest sardine caught, increasing the chances of catching new prey.

B. XGBoost

XGBoost is a powerful classification algorithm that started as a set of winning projects in a Kaggle competition. XGBoost is based on a gradient augmentation framework that continuously adds new decision trees to the fit to improve the efficiency and performance of the algorithm. XGBoost uses Taylor expansions to approximate the loss function, giving the model better trade-off bias and variance, and using fewer decision trees to obtain higher accuracy.

Suppose the given sample set has n samples and m features, it can be represented as

$$D = \{(x_i, y_i)\} \mid |D| = n, x_i \in R^m, y_i \in R \quad (4)$$

here is the feature value and is the actual value. The algorithm sums the results of the tree to the final predicted value, denoted as

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i) \quad f_k \in F \quad (5)$$

F is the set of decision trees:

$$F = \{f(x) = w_{q(x)}\} \quad (q: R^m \rightarrow T, w \in R^T) \quad (6)$$

Where $f(x)$ is one of the trees, is the weight of the leaf nodes. $W_{q(x)}$ is the number of leaf nodes, and q represents the structure of each tree, which maps the samples to the corresponding leaf nodes. Thus, the predicted value of XGBoost is the sum of the values of the leaf nodes of each tree. The goal of this model is to learn this K tree, so minimising the following objective function.

$$L^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (7)$$

Where m is the loss of the difference between the estimated value y_i and the actual value $\hat{y}_i$; common loss functions include the logarithmic loss function, the squared loss function and the exponential loss function. The use of regularization to set the penalty of the decision tree can prevent overfitting.

During the model training process, new trees are added to fit the residuals from the previous round. xGBoost performs a Taylor expansion of the objective function, taking the first three terms and removing the higher order small infinitesimal terms. The residuals between the predicted scores and do not affect the optimization of the objective function and are therefore removed. The algorithm converts the iterations of the tree model into iterations of the leaf nodes and calculates the optimal leaf node scores obtained. The final objective function is obtained by substituting the optimal value into the objective function as follows.

$$Obj = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T \quad (8)$$

C. Improve Sailfish Optimizer

In order to enhance the merit-seeking ability of the sailfish optimizer at the early stage of optimization, this paper adds a stochastic backward learning strategy initialization to improve it. In the sailfish optimization algorithm, the positive cosine strategy is added to update the sailfish position and sardine coefficients to increase the optimization-seeking ability of the sailfish optimizer. To increase the ability of the sailfish optimization algorithm to jump out of the local optimum, this paper adds that when the sailfish hunts sardines, the hunted sardines will use Levy flight to perform position update to achieve the ability of the algorithm to jump out of the local optimum. The improved sailfish optimization algorithm (ISFO) expands the population search range at initialization, exploits a new search space, and enhances the diversity of the population.

The ISFO pseudo-code is shown in Figure 1 below.

Algorithm 1: Improved sailfish optimization algorithm (ISFO)

Input: Initialization parameters, population number s , iteration number e , population proportion pp , problem dimension d , solution space range (L, R)

Step 1 s random sailfish and s/pp sardines were generated in the solution space, and the population initialization was completed by random inverse learning ;

Step 2 Calculate individual fitness value $F_{fitness}$. To get the best sailfish and sardines;

Step 3 The algorithm iterates and the position is updated to find the optimal value of the function

for The iteration e times **do**

 Step 3.1 The cosine strategy updates the sailfish position;

 Step 3.2 Calculate sailfish fitness;

 Step 3.3 Calculate sailfish slapping intensity AP ;

if $AP > habitat$ **then**

 Update all sardine positions;

else

 Some sardine positions updated;

 Step 3.4 Calculate the fitness of sardines;

 Step 3.5 The sailfish replaced the sardines in a more suitable position;

if $S_{fitness} > SF_{fitness}$ **then**

 Sailfish replace sardines;

 Levy Flight updates sardine position;

Step 4 Output the optimal fitness, the optimal sailfish position;

Figure 1. ISFO pseudo-code

During the initialization process, the fitness value of each sailfish as well as sardine, and the suitability value of each random reverse position of sailfish as well as sardine are calculated. Two symmetric regions are searched simultaneously, and the sailfish as well as sardine position with higher fitness is selected as the initialized position to improve the convergence efficiency and global search ability of the algorithm.

In random reverse learning, X_{old}^i is the position before the update and X_{new}^i is the position after the update. There exists a point X in the n dimensional coordinate system, while $X \in [LB, UB]$ and r represent random numbers between 0 and 1. The random reverse solution is calculated as follows:

$$X_{new}^i = LB + UB - rX_{old}^i \quad (9)$$

ISFO uses the cosine optimization strategy approach to select the position update towards the optimal position of the sailfish, which makes the algorithm more capable of local

search for superiority. The cosine optimisation strategy position update formula is as follows.

$$X_{new}^i = \cos(2\pi l) \times X_{best} - \lambda_i \times |2r \times X_{best} - X_{old}^i| \quad (10)$$

In the cosine optimization strategy X_{best} is the position of the best fit in the sailfish, λ_i is the coefficient of the iteration, l and r are random numbers between 0 and 1.

ISFO introduces a sinusoidal formulation of the AP coefficients for the update. where is the i iteration, e is total iterations, and r is a random number between 0 and 1.

$$AP = 1 - i \times \sin(2\pi r)/e \quad (11)$$

In SFO, when a sardine's position is better than the sailfish's position, the sailfish replaces that sardine's position. Here, we use Levi's Flight to perform a position update for that sardine after the sailfish has replaced the sardine position. The formula for the Lévy flight for position update is as follows.

$$X_{new}^i = X_{old}^i + l \oplus Levy(\alpha) \quad (12)$$

X_{old}^i is the position before the update, X_{new}^i is the position after the update, l denotes the weight of the control step, and levy flight denotes a path that obeys the Lévy distribution. The levy flight equation is as follows.

$$L(\alpha) = \frac{\mu}{|v|^{1-\alpha}}, \quad \mu \sim (0, \alpha_\mu^2), \quad v \sim (0, \alpha_v^2) \quad (13)$$

D. ISFO-XGBoost

As a powerful machine learning algorithm, XGBoost has a large number of hyperparameters. the ISFO can obtain the optimal values of multiple XGBoost hyperparameters in n -dimensional space at the same time. ISFO solves the problem of continuous optimization of the optimal hyperparameters of XGBoost by using the characteristics of the position update formula.

XGBoost has many parameters, and some of them have a great influence on the prediction effect of the model. The parameters that determine the effect of the model are mainly `n_estimators`, `max_depth`, `min_child_weight`, `learning_rate`, `subsample`, `colsample_bytree` and `gamma`. We control `n_estimators` to be 200 and select the other 6 parameters as the target parameters for optimization.

The ISFO based XGBoost (ISFO-XGBoost) as shown in Figure 2, which adaptively optimizes the parameters of the XGBoost algorithm during training, and then achieves the purpose of improving the overall performance of XGBoost classification.

The steps of the ISFO optimized XGBoost algorithm are as follows :

- 1) The dataset to be processed is read and the dataset is pre-processed.
- 2) The data set was divided into a training set and a test set, according to 7:3.
- 3) The fitness function of the ISFO optimization algorithm.

4) The ISFO optimisation algorithm is initialised and the XGBoost parameters are optimised iteratively to obtain the best parameters for the XGBoost model.

5) The optimised parameters are fed into the XGBoost model for fraud detection and the detection results are obtained.

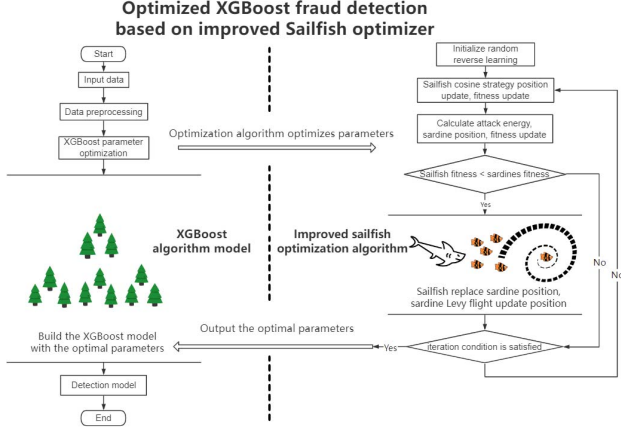


Figure 2. ISFO optimized XGBoost algorithm

III. RESULTS & DISCUSSION

The experimental environment was 12th Gen Intel(R) Core(TM) i7-12700H 2.30 GHz CPU, 16.00 GB RAM and NVIDIA GeForce RTX 3060 Laptop GPU, all programs were written and executed in python 3.9 and the operating system was Microsoft Windows 11.

A. Comparison of ISFO with other algorithms

We compared five other different population-based meta-heuristic algorithms for LSFO. The following algorithms were used: SFO, WOA, HHO and two other well-known PSO and GA, were compared and the efficiency of ISFO was measured. The parameters of the algorithms we used were combined with the actual algorithms and the values of the relevant parameters used for each algorithm are shown in Table 1.

TABLE I. OPTIMIZATION ALGORITHM PARAMETERS

algorithm	parameters
WOA	$a=(0,2), p = \text{rand}[0,1], b=1, l=(-1,1)$
HHO	$E0=(-1,1), J=(0,2)$
PSO	$w=(0.4,0.9), C1=C2=2$
GA	$pc=0.95, pm=0.25$
SFO	$pp=1, A=4, \text{epxilon}=0.001$
ISFO	$pp=1, l=\text{rand}[0,1]$

For testing the effectiveness, stability and robustness of these optimisation algorithms, we used a selection of functions from 23 benchmark functions, including single-peak, multi-peak and fixed-dimensional multi-peak functions. Each algorithm was executed 30 times. The dimensionality of the benchmark function was fixed at 100 to test its efficiency in high dimensional problems. The results were computed and tabulated to show the best values, see Table 2. the best scale-hacks in the table show that SFO performed best in F5 and F11, HHO in F6 as well as PSO in F13, and the rest of the tested functions performed very well.

TABLE II. TABLE OF RESULTS FOR FUNCTIONS

function	ISFO	SFO	WOA	HHO	PSO	GA
$f_1(x) = \sum_{i=1}^n x_i^2$	0	3.46e-63	3.92e-95	6.402e-6	3.51e+03	4.07e+03
$f_2(x) = \sum_{i=1}^n x_i + \prod_{i=1}^n x_i $	2.11e-26	5.22e-53	1.42e-54	1.26e-41	1.18e+03	4.36e+02
$f_3(x) = \sum_{i=1}^n \left(\sum_{j=1}^i x_j^2 \right)^2$	0	3.088e-1	2.14e+01	5.31e+05	9.26e+03	1.26e+04
$f_4(x) = \max_{1 \leq i \leq n} x_i $	3.354e-2	1.80e-09	6.63e-16	4.15e-15	2.35e+01	1.86e+01
$f_5(x) = \sum_{i=1}^n \left[100(x_{i-1} - x_i^2)^2 + (x_i - 1)^2 \right]$	9.55e-08	7.33e-08	2.61e-03	3.74e-06	6.48e+04	5.69e+04
$f_6(x) = \sum_{i=1}^n \left[(x_i + 1)^2 \right]$	2.29e-01	2.11e+00	5.97e-03	4.21e-05	4.50e+03	3.68e+03
$f_7(x) = \sum_{i=1}^n x_i^4 + \text{random}(0,1)$	2.63e-06	5.89e-05	1.96e-04	6.89e-05	1.20e+00	1.20e-01
$f_8(x) = \sum_{i=1}^n -x_i \sin(\sqrt{ x_i })$	0	0	0	0	5.27e+02	7.38e+02
$f_9(x) = -20 \exp \left(-0.2 \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2} \right) - \exp \left(\frac{1}{n} \sum_{i=1}^n \cos(2\pi x_i) \right) + 20 + e$	4.44e-16	2.17e-14	3.99e-15	3.99e-15	1.07e+01	9.51e+00
$f_{10}(x) = \frac{1}{4000} \sum_{i=1}^n x_i^2 - \prod_{i=1}^n \cos \left(\frac{x_i}{\sqrt{i}} \right) + 1$	0	0	0	0	4.74e+01	3.67e+01
$f_{11}(x) = 0.1 \left[\sin^2(3\pi x_1) + \sum_{i=2}^n (x_i - 1)^2 \frac{1}{i^2} + \sin^2(3\pi x_1 + 1) + (x_n - 1)^2 \frac{1}{n^2} + \sin^2(2\pi x_n) \right] + \sum_{i=1}^n i(x_i - 1.5, 100, 4)$	1.69e-07	2.91e-08	9.09e-03	4.87e-05	2.28e+03	3.67e+01
$f_{12}(x) = \sum_{i=1}^{41} \left[a_i - \frac{x_i(b_i^2 + b_i x_i)}{b_i^2 + b_i x_i + 4} \right]^2$	3.16e-04	3.07e-04	3.39e-04	3.07e-04	3.07e-04	4.58e-03

The convergence speed comparison is shown in Figure 3, which demonstrates that ISFO can find the best solution among other competing algorithms early in the iterative process. The strong search ability of ISFO early on can be seen in the

convergence speed graph, a performance attributed to the cosine function optimising the sailfish position, pulling the other members of the sailfish population towards the optimal solution in the first few iterations. The behaviour of the sardine

is important during the optimisation process, as the flagfish will occupy the position when a good individual emerges from the sardine, and that sardine will levy flight so that it goes to jump out of the local optimum.

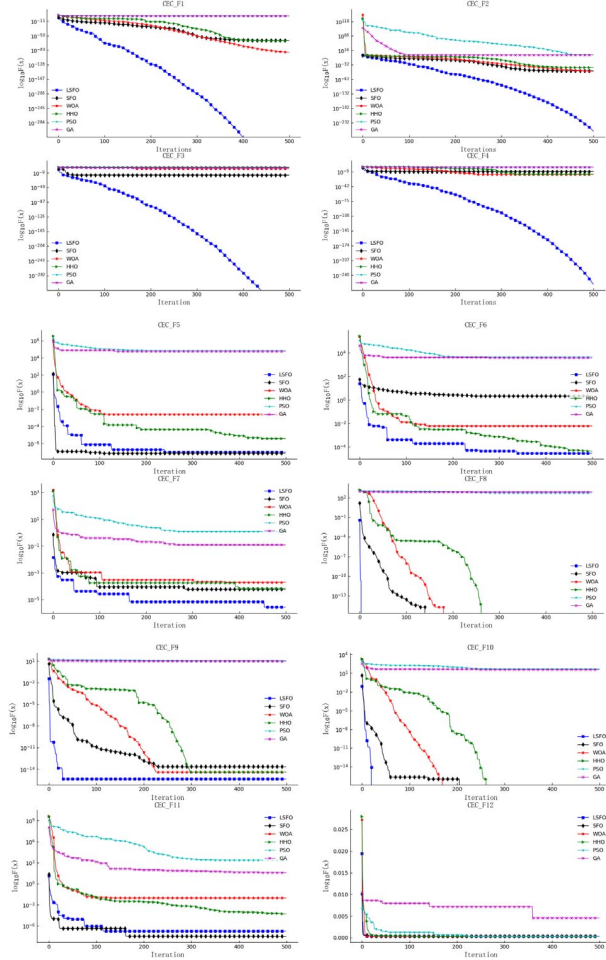


Figure 3. Graph of convergence speed of algorithm

B. Comparison of ISFO-XGBoost with other algorithms

The dataset chosen for the experiments was creditcard.csv on kaggle, and the model was constructed in XGBoost, random forest and decision tree. Table 3 lists the Accuracy, Precision, Recall, AUC and F1-Score of this dataset for several anomaly detection algorithms used to evaluate the performance of the model. The fitness functions were defined using weights of 0.1, 0.1, 0.3, 0.2, and 0.3 for each optimisation algorithm, focusing on the Recall, AUC and F1-Score results.

As can be seen from table, in the five performance evaluation metrics, ISFO-XGBoost wins with Accuracy of 0.999566, Precision of 0.841379, Recall of 0.897058, AUC of 0.948394 and F1-Score of 0.868327.

TABLE III. COMPARISON OF FRAUD DETECTION ALGORITHMS

Detection algorithm	Precision	Recall	Accuracy	AUC	F1-Score
DT	0.274038	0.838235	0.996207	0.917347	0.413043
Random forest	0.721212	0.875	0.999262	0.937230	0.790697
XGBoost	0.790540	0.860294	0.999414	0.929965	0.823943
SFO-XGBoost	0.839160	0.882352	0.999543	0.941041	0.860215
Bayesian-XGBoost	0.809523	0.875	0.999473	0.937335	0.840989
GA-XGBoost	0.818181	0.860294	0.999473	0.929994	0.838709
PSO-XGBoost	0.840277	0.889705	0.999555	0.944718	0.864285
ISFO-XGBoost	0.841379	0.897058	0.999566	0.948394	0.868327

IV. CONCLUSION

XGBoost is based on GBDT, to which regularisation constraints are added, loss functions are improved, etc. It has good detection results in the field of fraud detection, but the structure of XGBoost model relies on the setting of its hyperparameters. In this paper, we study the problem of fraud detection in online transactions and propose the use of SMOTE as pre-processing to solve data imbalance in order to improve the accuracy of the model. In using XGBoost as the fraud detection algorithm, a modified sailfish optimiser is used as the optimiser to optimise the hyperparameters of XGBoost. Our improved sailfish optimiser chooses 12 classical test functions and achieves good results in comparison with other classical optimisation algorithms. In fraud detection, ISFO-XGBoost is superior to other models in fraud detection when comparing some common classification methods by Accuracy, Precision, Recall, AUC and F1-Score performance metrics. In comparison with SFO, Bayesian, GA and PSO optimised XGBoost, it was found that ISFO-XGBoost has good results on the creditcard dataset, with Recall and AUC being more effective. In future work, various models are considered to handle online transaction fraud detection, including machine learning, artificial intelligence and deep learning approaches, where there is scope for developing more accurate and efficient models. The current research could be improved from several perspectives. Hybrid models can be built by combining two or more models to improve the research approach. In addition, the detection algorithms will be tested on large amounts of real-time data.

REFERENCES

- [1] Roberston D (2021) The Nelson Report. Available online: https://nilsonreport.com/content_promo.php?id_promo=16 [Last accessed: 18 March 2021]
- [2] Maniraj S P, Saini A, Ahmed S, et al. Credit card fraud detection using machine learning and data science[J]. International Journal of Engineering Research, 2019, 8(9): 110-115.
- [3] S. Khatri, A. Arora and A. P. Agrawal, "Supervised machine learning algorithms for credit card fraud detection: A comparison", Proc. 10th Int. Conf. Cloud Comput. Data Sci. Eng. (Confluence), pp. 680-683, Jan. 2020.
- [4] N. K. Trivedi, S. Simaiya, U. K. Lilhore and S. K. Sharma, "An efficient credit card fraud detection model based on machine learning methods", Int. J. Adv. Sci. Technol., vol. 29, no. 5, pp. 3414-3424, 2020
- [5] S. Rajora, D. L. Li, C. Jha, N. Bharill, O. P. Patel, S. Joshi, et al., "A comparative study of machine learning techniques for credit card fraud detection based on time variance", Proc. IEEE Symp. Comput. Intell. (SSCI), pp. 1958-1963, Nov. 2018.

- [6] Dornadula V N, Geetha S. Credit card fraud detection using machine learning algorithms[J]. *Procedia computer science*, 2019, 165: 631-641.
- [7] Ensor K B, Glynn P W. Stochastic optimization via grid search[J]. *Lectures in Applied Mathematics-American Mathematical Society*, 1997, 33: 89-100.
- [8] Bergstra J, Bengio Y. Random search for hyper-parameter optimization[J]. *Journal of machine learning research*, 2012, 13(2).
- [9] Putatunda S, Rama K. A comparative analysis of hyperopt as against other approaches for hyper-parameter optimization of XGBoost[C]//*Proceedings of the 2018 International Conference on Signal Processing and Machine Learning*. 2018: 6-10.
- [10] Qin C, Zhang Y, Bao F, et al. XGBoost optimized by adaptive particle swarm optimization for credit scoring[J]. *Mathematical Problems in Engineering*, 2021, 2021.
- [11] Chen T, He T, Benesty M, et al. Xgboost: extreme gradient boosting[J]. *R package version 0.4-2*, 2015, 1(4): 1-4.
- [12] R. Saia and S. Carta, "Evaluating credit card transactions in the frequency domain for a proactive fraud detection approach", *Proc. 14th Int. Joint Conf. e-Bus. Telecommun.*, vol. 4, pp. 335-342, 2017.
- [13] Shadravan S, Naji H R, Bardsiri V K. The Sailfish Optimizer: A novel nature-inspired metaheuristic algorithm for solving constrained engineering optimization problems[J]. *Engineering Applications of Artificial Intelligence*, 2019, 80: 20-34.